

HOUSE PRICE PREDICTION: A DATA-DRIVEN STUDY OF THE FACTORS INFLUENCING RESIDENTIAL PROPERTY VALUES USING REGRESSION MODELS

**Project Report Submitted in fulfillment of the
Requirements for the Award of the Internship of
Summer Training Program by IDRA**

Data Science and AI

By:

Name: JAMUNA MOHAN

Institute: KONGU ENGINEERING COLLEGE

Institute Roll No.: 24CSR109

Enrollment No.: IDRA-2026-306450

Under the Supervision of

Dr. Shaheena Salam

Through the platform:



India Data Research Academy

DECLARATION

I hereby declare that this project report, "House Price Prediction: A Data-Driven Study of the Factors Influencing Residential Property Values Using Regression Models", is submitted as part of the IDRA Data Science & AI training programme. The analysis is based on the dataset provided by IDRA through the programme's learning management system, and the accompanying notebook reproduces every result reported here. I have reviewed the code and results and I am able to explain each step. AI-based tools were used to assist with coding and drafting; all outputs were checked by me. All external sources, libraries and datasets used are acknowledged in the References.

ACKNOWLEDGEMENT

I sincerely thank India Data Research Academy for the training programme and for providing the dataset and the capstone framework used in this project. I am grateful to my supervisor, Dr. Shaheena Salam, for her guidance, and to [Institute Name] for its support. I also thank the developers of the open-source Python libraries used in this analysis, and my peers and family for their encouragement.

ABSTRACT

House prices depend on a mix of size, quality and location characteristics, yet buyers, sellers and lenders often lack a quick, evidence-based way to judge what a home should cost. This project analyses 545 houses described by 12 features (area, bedrooms, bathrooms, stories, parking, six yes/no amenities and furnishing status) to explain and predict sale price. The data contained no missing values or duplicates, so cleaning focused on validation and type conversion; 15 price outliers and 12 area outliers were retained as genuine high-end properties. Price is right-skewed (skewness 1.21) and is most strongly correlated with area ($r = 0.54$), bathrooms ($r = 0.52$) and air conditioning ($r = 0.45$). Six regression models were compared using five-fold cross-validation on an 80% training split. A linear regression on log-transformed price, using log-area, performed best in validation and explained 65.9% of price variance on the unseen test set (MAE 0.97 million, about 20% of the average price; RMSE 1.31 million), with a small train-test gap (R^2 0.70 versus 0.66). Tree-based ensembles overfit and did not improve on it. The model tends to under-predict the most expensive homes. Bathrooms, air conditioning, preferred-area location and area emerged as the main price drivers. The model suits rough valuation and screening, not the pricing of individual transactions.

Keywords: house price prediction; regression; exploratory data analysis; data preprocessing; feature engineering; predictive analytics

TABLE OF CONTENTS

1. Introduction.....	5
2. Problem Statement and Motivation.....	5
3. Objectives.....	5
4. Research Questions.....	6
5. Dataset Description and Data Understanding.....	6
6. Data Cleaning.....	7
7. Data Preprocessing.....	8
8. Exploratory Data Analysis.....	9
9. Statistical Analysis.....	13
10. Feature Engineering.....	14
11. Model Development.....	14
12. Model Evaluation.....	15
13. Results and Findings.....	19
14. Discussion.....	19
15. Limitations.....	20
16. Conclusion.....	20
17. Recommendations and Future Work.....	20
18. References.....	21
Appendix A. Code.....	21
Appendix B. Additional Figures and Tables.....	24

1. INTRODUCTION

1.1 Background

A home is the largest financial asset most households will ever own, and its price influences decisions made by buyers, sellers, banks, insurers, developers and local planners. Yet pricing a house is difficult: the value of a property reflects its size, the number and type of rooms, the amenities it offers, how it is furnished, and where it sits relative to roads and desirable neighbourhoods. Traditionally, prices are estimated from comparable recent sales and the judgement of an experienced appraiser. That approach is slow, hard to scale, and can vary from one appraiser to another. Data-driven models offer a consistent, transparent baseline that can be produced instantly and audited.

1.2 Context of the Problem

This project studies a cross-sectional dataset of 545 residential properties, each described by its area, room counts, number of storeys, parking capacity, a set of yes/no amenities (main-road access, guest room, basement, hot-water heating, air conditioning, preferred-area location) and furnishing status, together with its price. The likely users of an analysis like this are property agents setting or checking asking prices, buyers judging whether a listing is fairly priced, lenders screening collateral, and developers deciding which features are worth building. In each case two questions matter: how accurately can price be predicted from a property's recorded characteristics, and which characteristics carry the most weight?

1.3 Scope of the Study

The study covers the full data-science workflow on the provided dataset: quality checks and cleaning, preprocessing, exploratory and statistical analysis, feature engineering, and the development and evaluation of regression models that predict price. The report structure follows the IDRA Capstone Project Guide (India Data Research Academy [IDRA], 2026a). The study does **not** cover market trends over time (the data has no dates), detailed neighbourhood effects (only two rough location indicators exist), property age or condition (not recorded), causal effects of individual features, or deployment of the model as a live tool.

2. PROBLEM STATEMENT AND MOTIVATION

Problem statement: Using the physical and amenity characteristics recorded for 545 residential properties, how accurately can the price of a house be predicted, and which characteristics are most strongly associated with higher prices?

Answering this has practical value. A validated model gives a fast, objective reference price and helps separate features that genuinely move prices from those that only appear to. It also shows where data alone is not enough. Knowing where a model fails is as important as knowing how well it does on average, because it tells a user when to trust an estimate and when to seek an expert valuation.

3. OBJECTIVES

- To assess the quality of the housing dataset and prepare it for analysis without discarding valid observations.
- To identify the property characteristics most strongly associated with price using exploratory analysis, correlation and group comparisons.
- To build and compare regression models that predict house price, selecting the final model on cross-validated error rather than on the test set.
- To evaluate the selected model on unseen data using MAE, MSE, RMSE and R^2 , and to check for overfitting by comparing training and test performance.

- To turn the findings into practical, evidence-based recommendations while stating clearly what the model cannot do.

4. RESEARCH QUESTIONS

1. What are the main characteristics and data-quality issues of the housing dataset?
2. Which property features are most strongly associated with price, and does price differ systematically across groups such as air-conditioned versus non-air-conditioned homes or preferred versus other areas?
3. How accurately can a regression model predict price from these features, and does a more flexible model improve on a simple one?
4. Where does the model make its largest errors, and what does that imply about how it should be used?

5. DATASET DESCRIPTION AND DATA UNDERSTANDING

5.1 Data Source

The dataset was provided by IDRA through the Study Material section of the programme's learning management system as the file `P_9_Housing.csv` for Project 9 (House Price Prediction) (IDRA, 2026b). The file itself carries no documentation of how the data was collected, which city or region it covers, when the sales occurred, or the currency and area unit used. Prices and areas are therefore analysed in the dataset's own units, and price figures in this report are quoted in millions of those units. This is discussed further under Limitations.

The analysis was implemented in Python using pandas (McKinney, 2010), NumPy (Harris et al., 2020) and scikit-learn (Pedregosa et al., 2011); figures were produced with Matplotlib (Hunter, 2007) and seaborn (Waskom, 2021).

5.2 Dataset Size and Structure

The dataset contains **545 observations and 13 variables** (Table 1). Each row is one property. The target variable is `price`, a continuous quantity, which makes this a regression problem. The other twelve variables are candidate predictors: five numeric (`area`, `bedrooms`, `bathrooms`, `stories`, `parking`), six yes/no indicators and one three-level category (`furnishingstatus`).

Attribute	Value
Observations (rows)	545
Variables (columns)	13 (12 predictors + 1 target)
Target variable	<code>price</code> (continuous), so the task is regression
Numeric variables	6: <code>price</code> , <code>area</code> , <code>bedrooms</code> , <code>bathrooms</code> , <code>stories</code> , <code>parking</code>
Yes/no variables	6: <code>mainroad</code> , <code>guestroom</code> , <code>basement</code> , <code>hotwaterheating</code> , <code>airconditioning</code> , <code>prefarea</code>
Categorical variable	1: <code>furnishingstatus</code> (furnished, semi-furnished, unfurnished)
Missing values / duplicate rows	0 / 0
Price (mean, median, range)	4.77M, 4.34M, 1.75M to 13.30M

Table 1. Summary of the Dataset

5.3 Data Dictionary

Table 2 lists every variable. Meanings are taken from the column names, since the file has no separate documentation.

Variable	Meaning	Type	Role
price	Sale price of the house (currency not stated)	Numeric, continuous	Target
area	Area of the property (unit not stated)	Numeric, 1,650 to 16,200	Feature
bedrooms	Number of bedrooms	Count, 1 to 6	Feature
bathrooms	Number of bathrooms	Count, 1 to 4	Feature
stories	Number of storeys	Count, 1 to 4	Feature
mainroad	House is connected to a main road	Binary (yes/no)	Feature
guestroom	House has a guest room	Binary (yes/no)	Feature
basement	House has a basement	Binary (yes/no)	Feature
hotwaterheating	House has hot-water heating	Binary (yes/no)	Feature
airconditioning	House has air conditioning	Binary (yes/no)	Feature
parking	Number of parking spaces	Count, 0 to 3	Feature
prefarea	House is in a preferred area	Binary (yes/no)	Feature
furnishingstatus	Furnished, semi-furnished or unfurnished	Categorical (3 levels)	Feature

Table 2. Data Dictionary

6. DATA CLEANING

Every check below was run on the raw file first, so that this report can state honestly what was and was not wrong with the data. Cleaning actions were taken only where a genuine problem was found; altering clean data would only introduce error.

6.1 Missing Values

No column contains missing values: 0 of 7,085 cells are empty. No imputation or row deletion was therefore needed.

6.2 Duplicate Records

There are 0 fully duplicated rows, so no rows were removed and the dataset size is unchanged at 545 observations. Many homes share identical prices, but a repeated price alone does not make a duplicate record.

6.3 Data Types and Inconsistent Values

Seven columns are stored as text. All seven were checked for stray spaces and mixed capitalisation, which would silently split one category into several; **0 inconsistencies** were found. The six yes/no columns contain exactly two labels and `furnishingstatus` exactly three. Numeric columns were checked for impossible values: price, area, bedrooms, bathrooms and stories are all strictly positive, and `parking` is never negative (zero simply means no parking space). As a further sanity check, price per unit of area ranges from 270 to 2,640 with a median of 952, so no home has an implausible price-to-size combination. The one real conversion was to store the seven text columns as categorical variables so they could be encoded consistently in the next stage.

```
missing = raw.isna().sum()
label_issues = int(sum((raw[c] != raw[c].str.strip().str.lower()).sum() for c in
cat_cols))
invalid = int((raw[positive_cols] <= 0).sum().sum() + (raw["parking"] < 0).sum())
```

6.4 Outlier Handling

Outliers were checked on the two continuous variables with the standard IQR rule (values beyond $Q3 + 1.5 \times IQR$). Table 3 shows that 15 homes (2.8%) are unusually expensive and 12 homes (2.2%) unusually large. The count variables take only small whole-number values (1 to 6 bedrooms, for example) and were not treated as outlier candidates.

Variable	Upper fence	Homes above fence	Share of rows	Largest value	Decision
price	9,205,000	15	2.8%	13,300,000	Retain
area	10,500	12	2.2%	16,200	Retain

Table 3. Outlier Check Using the IQR Rule

Decision: the outliers were retained. They are large or high-end homes, not data-entry errors: their price-per-area values fall inside the normal range and none has an impossible combination of price and size. Real luxury properties are exactly the cases a valuation model must eventually handle, and with only 545 rows, deleting 15 to 25 of them would discard scarce information. Instead of removing them, the skew they create is addressed by a log transformation (Section 10).

6.5 Before vs After Cleaning

Table 4 compares the data before and after cleaning. Because the raw data was already sound, the only change is in data types; the value of these checks is the documented evidence that the data is fit for modelling.

Check	Before	After	Comment
Rows	545	545	No rows removed
Missing cells	0	0	None found
Duplicate rows	0	0	None found
Label inconsistencies	0	0	Labels already consistent
Impossible values	0	0	All values within valid ranges
Columns stored as text	7	0	Converted to categorical
Columns stored as categorical	0	7	Ready for encoding

Table 4. Data Quality Before and After Cleaning

7. DATA PREPROCESSING

Preprocessing converts the cleaned data into the numeric form that machine-learning models require. The order matters: steps that follow a fixed rule and learn nothing from the data are done before the train-test split; steps that learn parameters from data are done only after it, and fitted on the training data alone.

7.1 Categorical Encoding

The six yes/no variables were mapped to 1 and 0. `furnishingstatus` has three unordered levels, so it was one-hot encoded into two indicator columns, with 'furnished' as the reference level (dropping one level avoids perfectly redundant columns). These are fixed rules that learn nothing from the data, so applying them before the split cannot leak information.

```
data = df.copy()
for c in binary_cols:
    data[c] = (data[c] == "yes").astype(int)
data = pd.get_dummies(data, columns=["furnishingstatus"], drop_first=True, dtype=int)
```

7.2 Feature Scaling and Transformation

Linear models are sensitive to the scale of their inputs, so features are standardised (mean 0, standard deviation 1) using `StandardScaler` **inside a pipeline**. Within cross-validation the scaler is re-fitted on each training fold, and for the final model it is fitted on the training set only; the test set is only ever transformed, never used to fit. Tree-based models do not need scaling and were not scaled. Because price is right-skewed, one linear model is trained on $\log(\text{price})$, with predictions converted back to price units using the exponential function so that all metrics are comparable.

7.3 Train-Test Split

The data was split 80% / 20% with a fixed random seed of 42, giving **436 training and 109 test observations**. The split is needed to measure performance on homes the model has never seen. To avoid data leakage, the test set was not used for any decision: feature choices and model selection relied on five-fold cross-validation within the training set, and the test set was scored once at the end. The dataset contains no column that reveals the price directly or that would only be known after a sale, so no leakage columns had to be removed.

```
feature_cols = [c for c in data.columns if c not in ("price", "amenity_score")]
X, y = data[feature_cols].copy(), data["price"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=RANDOM_STATE)
```

7.4 Final Feature Set

The final model uses **13 input features** and one target. The features are: bedrooms, bathrooms, stories, mainroad, guestroom, basement, hotwaterheating, airconditioning, parking, prefarea, furnishingstatus_semi-furnished, furnishingstatus_unfurnished, log_area. The target is price. Note that log_area replaces area (Section 10).

8. EXPLORATORY DATA ANALYSIS

The exploratory analysis is organised around the research questions: what does the target look like, which features move with it, and how should those observations shape the model? Prices are plotted in millions for readability. Only the charts that support a conclusion are shown here; further charts are in Appendix B.

8.1 Univariate Analysis

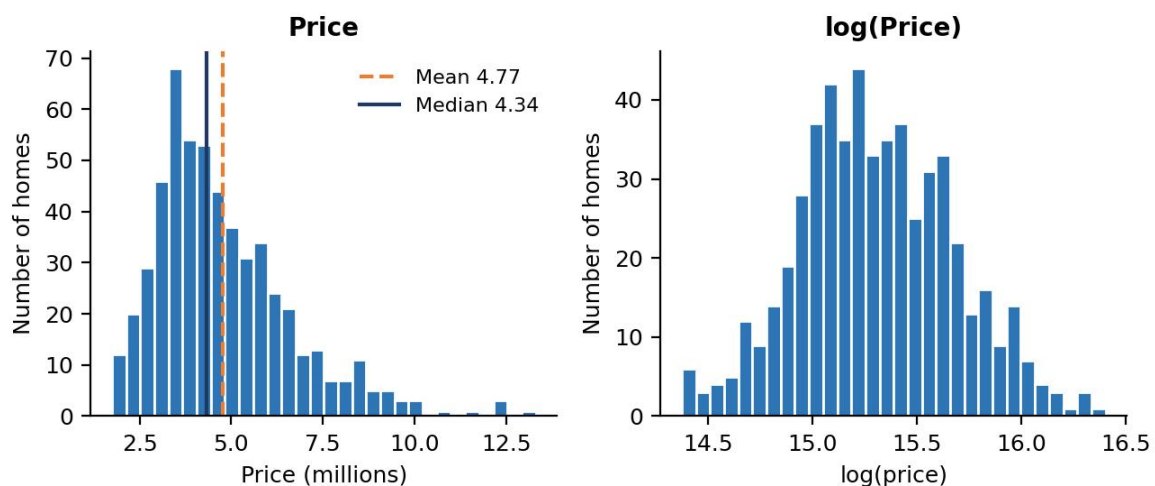


Figure 1. Distribution of House Price (left) and Log Price (right)

Figure 1 shows that price is **right-skewed**: the middle half of homes lie between about 3.4 and 5.7 million, while a smaller number of expensive homes stretch the right tail. The mean (4.77M) sits above the median (4.34M) and skewness is 1.21. After a log transformation the distribution is close to symmetric (skewness 0.14). This is the first reason to consider modelling $\log(\text{price})$, and it makes the median a fairer description of a typical home than the mean.

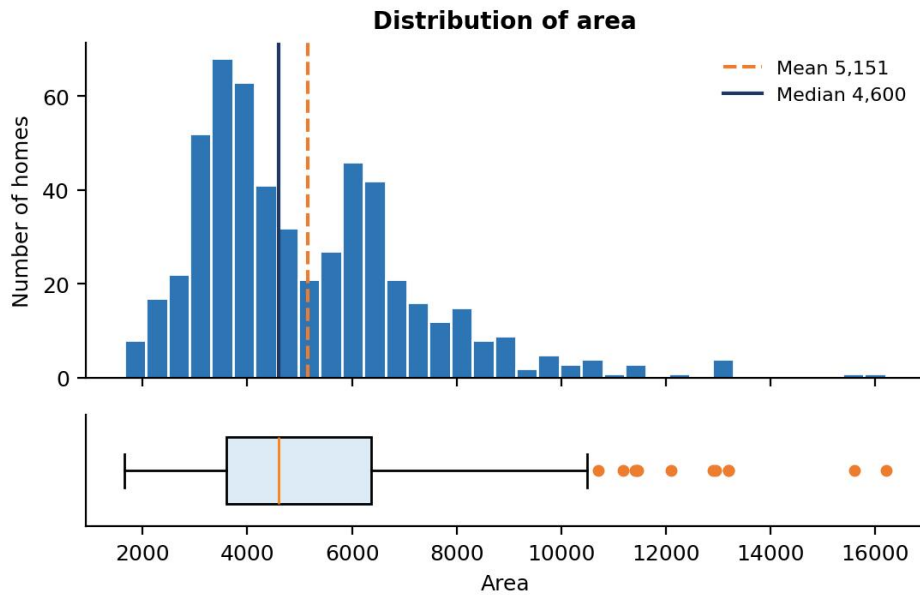


Figure 2. Distribution of Property Area with Box Plot

Area shows the same pattern (Figure 2): mean 5,151, median 4,600, skewness 1.32, and 12 large homes flagged beyond 10,500. Its skewness falls to 0.13 after taking logs, which motivates the $\log_area$ feature tested in Section 10.

8.2 Categorical Analysis

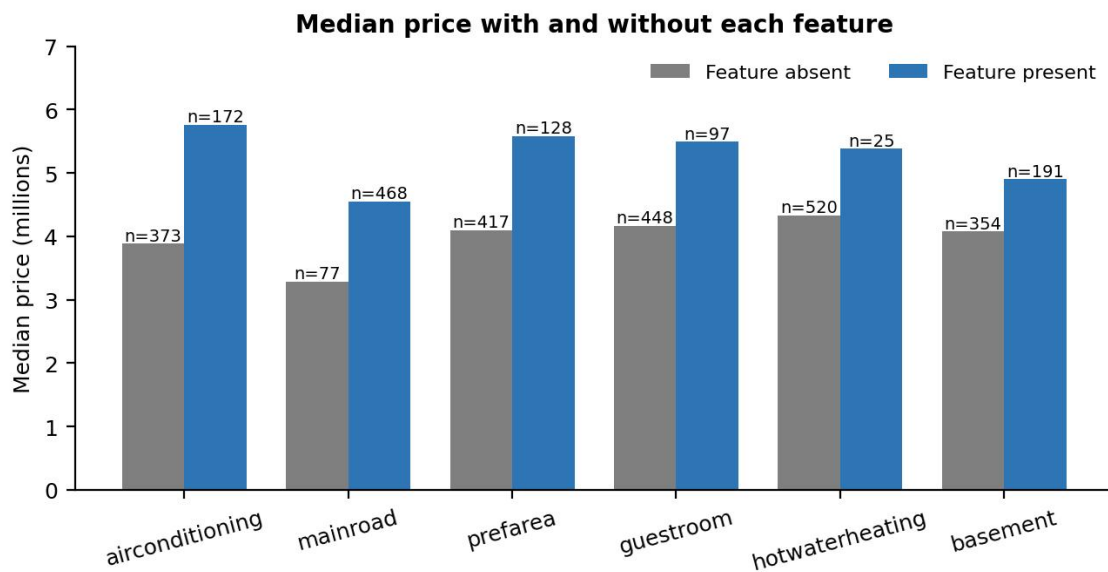


Figure 3. Median Price With and Without Each Yes/No Feature

Homes with each feature have a higher median price than homes without it (Figure 3). The largest raw gap is for air conditioning (median 48% higher), followed by main-road access (38%), preferred area (36%), guest room (32%), hot-water heating (24%) and basement (20%). These are simple comparisons that ignore other differences between the groups, so they overstate the effect of any single feature. For example,

air-conditioned homes also tend to be larger and have more bathrooms. Section 12.4 separates these effects. The hot-water group is small (25 homes), so its estimate is less reliable.

Median price by property characteristic

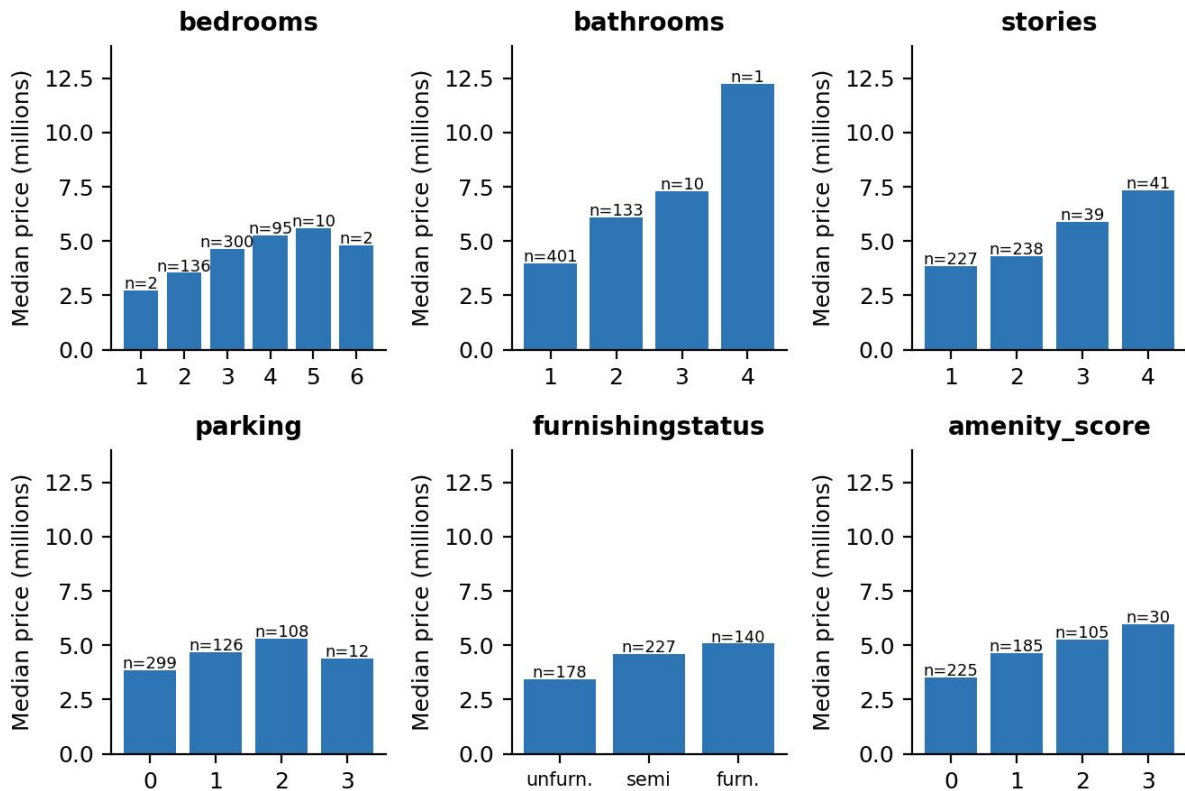


Figure 4. Median Price by Room Counts, Parking, Furnishing Status and Amenity Count

Figure 4 shows how price varies with size and quality indicators. Median price rises steadily with bathrooms (3.99M for one, 6.09M for two, 7.32M for three) and with storeys (3.85M for one, 7.35M for four). Bedrooms show a weaker trend (3.54M for two, 5.25M for four). Parking increases up to two spaces and then dips at three, but only 12 homes have three spaces. Furnished homes are the most expensive (median 5.08M) and unfurnished the least (3.43M). The composite amenity count (guest room, basement, hot-water heating and air conditioning) is also monotonic, from 3.50M with none to 5.99M with three. Groups with very few homes (one or six bedrooms, four bathrooms) are annotated with their counts and should not be over-interpreted.

8.3 Bivariate / Relationship Analysis

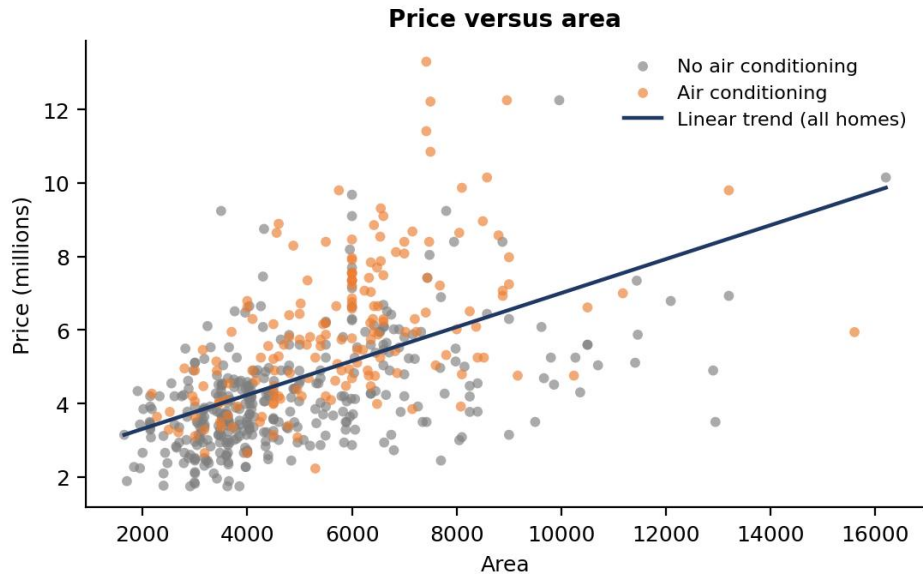


Figure 5. Relationship Between Area and Price, Coloured by Air Conditioning

Figure 5 shows a clear positive relationship between area and price ($r = 0.54$), but with wide scatter: area alone explains only about 29% of the variation in price ($r^2 = 0.29$). Two further patterns are visible. Air-conditioned homes (orange) sit above non-air-conditioned homes of similar size, so other features clearly matter alongside area. And the vertical spread of prices widens as area grows, meaning errors will naturally be larger for bigger, pricier homes. This unequal spread is another argument for modelling on a log scale.

8.4 Correlation Analysis

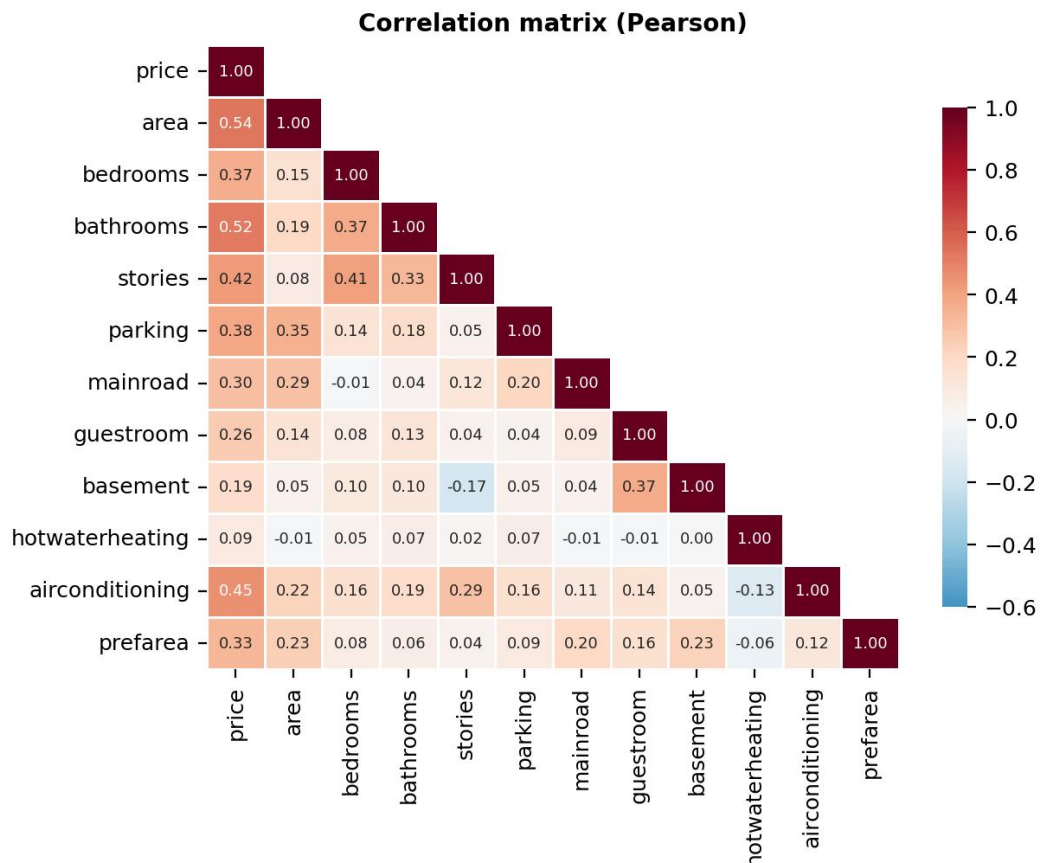


Figure 6. Correlation Heatmap of Price and Numeric/Binary Variables

All correlations with price are positive (Figure 6). The strongest are area (0.54), bathrooms (0.52), air conditioning (0.45), stories (0.42), parking (0.38) and bedrooms (0.37); hot-water heating is weakest (0.09). Among the predictors themselves the highest correlation is between bedrooms and stories (0.41), which is moderate, so there is no serious multicollinearity that would make a linear model unstable. Correlation describes association only; it does not show that adding a bathroom or an air conditioner would by itself raise a home's price.

8.5 Other Figures

Box plots of price for every yes/no feature, which show the spread and outliers behind the medians in Figure 3, are provided in Appendix B (Figure 10).

9. STATISTICAL ANALYSIS

Table 5 summarises the numeric variables with the measures of centre and spread covered in the programme. Price is shown in millions.

Variable	Mean	Median	Mode	Std. dev.	IQR	Skewness
price (millions)	4.77	4.34	3.50	1.87	2.31	1.21
area	5,151	4,600	6,000	2,170	2,760	1.32
bedrooms	2.97	3.00	3.00	0.74	1.00	0.50
bathrooms	1.29	1.00	1.00	0.50	1.00	1.59
stories	1.81	2.00	2.00	0.87	1.00	1.08
parking	0.69	0.00	0.00	0.86	1.00	0.84

Table 5. Descriptive Statistics of Key Variables

Price has a mean of 4.77M but a median of 4.34M, and its skewness of 1.21 is well above zero, so the median is the more representative centre. The standard deviation is 1.87M (variance 3.50e+12, its square) and the middle half of homes span an IQR of 2.31M. Area is similarly skewed (skewness 1.32). The count variables are tightly clustered: the typical home has 3 bedrooms, 1 bathroom, 2 storeys and no parking space (the modal values).

Table 6 gives the exact group medians behind Figure 3.

Feature	Homes with feature	Median without (M)	Median with (M)	Difference in medians
airconditioning	172	3.88	5.76	+48.2%
mainroad	468	3.29	4.55	+38.3%
prefarea	128	4.09	5.58	+36.3%
guestroom	97	4.17	5.50	+31.9%
hotwaterheating	25	4.33	5.38	+24.3%
basement	191	4.08	4.90	+20.2%

Table 6. Median Price by Presence of Each Feature

Every feature is associated with a higher median price. As noted above, these unadjusted differences mix the effect of the feature with the effect of everything that tends to accompany it, so they are descriptive, not causal.

10. FEATURE ENGINEERING

Four new variables were tested. The rule was fixed in advance: a candidate is kept only if it lowers the five-fold cross-validated RMSE of a linear model on log-price, fitted on the training data alone. The test set played no part. `log_area` is the natural logarithm of area, which tames the skew seen in Figure 2 and lets price respond in proportion to size. `amenity_score` counts the four premium amenities (guest room,

basement, hot-water heating, air conditioning). `area_per_bedroom` and `bath_per_bedroom` are ratios intended to capture spaciousness.

Feature set	CV RMSE (M)	Change vs baseline	Decision
Baseline (13 original features)	1.005	-	Reference
+ <code>log_area</code>	0.995	-1.04%	Improves; replacing is better
<code>log_area</code> replaces <code>area</code>	0.988	-1.75%	Adopted
+ <code>amenity_score</code>	1.005	+0.00%	Rejected: redundant
+ <code>area_per_bedroom</code>	1.008	+0.24%	Rejected: no gain
+ <code>bath_per_bedroom</code>	1.008	+0.29%	Rejected: no gain

Table 7. Cross-Validated Effect of Candidate Engineered Features

Replacing `area` with `log_area` gave the lowest error (0.988M, 1.75% below the baseline) and was adopted (Table 7). `amenity_score` changes nothing because a linear model can already add up its four components, which makes it redundant, and the two ratio features did not help. The improvement from `log_area` is modest, but it is principled (it addresses a documented skew), and it costs nothing in complexity. Together with the encoding in Section 7, this gives the 13-feature final set.

11. MODEL DEVELOPMENT

11.1 Problem Formulation

The task is **regression**. The target y is the house price (a continuous variable) and the input matrix x holds the 13 features listed in Section 7.4. Regression is the right formulation because the goal is to estimate a price on a continuous scale, not to assign a class.

11.2 Model Selection

Six models were compared, from the simplest possible baseline to flexible tree ensembles (Table 8). Starting simple is deliberate: with only 436 training rows, a simple model may generalise as well as a complex one, and it is far easier to interpret. The mean baseline shows the minimum any useful model must beat. The **selection rule was fixed before looking at any test result**: the final model is the one with the lowest five-fold cross-validated RMSE on the training data.

Model	Key settings	Purpose
Mean baseline	Predicts the training mean	Minimum bar any model must beat
Linear Regression	Standardised features; price as target	Simple, interpretable
Linear Regression (log price)	Standardised features; <code>log(price)</code> target, back-transformed	Handles skewed target
Random Forest (default)	100 trees, default depth	Flexible; expected to overfit on 436 rows
Random Forest (depth-limited)	300 trees, max depth 6, min 5 samples per leaf	Same model with overfitting controls
Gradient Boosting	Defaults: 100 trees, learning rate 0.1, depth 3	Sequentially built trees

Table 8. Models Compared and Their Settings

11.3 Training the Model

All models use the scikit-learn workflow (Pedregosa et al., 2011): fit on the training set, predict, evaluate. Cross-validation uses five shuffled folds with seed 42. The pipeline below shows how the linear model is built: scaling and the optional log transformation of the target are packaged with the regression so that every fit uses training data only.

```
def make_linear(log_target):
    pipe = make_pipeline(StandardScaler(), LinearRegression()) # scaler is fitted inside
    # each CV fold / on train only
```

```

if log_target:
    return TransformedTargetRegressor(regressor=pipe, func=np.log,
    inverse_func=np.exp)
return pipe

```

```

for name, m in models.items():
    cv = cv_rmse(m, X_train_f, y_train)          # cross-validation on TRAINING data only
    m.fit(X_train_f, y_train)
    tr, te = metrics(m, X_train_f, y_train), metrics(m, X_test_f, y_test)

```

12. MODEL EVALUATION

Four regression metrics are reported. **MAE** (mean absolute error) is the average size of a prediction error in price units. **MSE** (mean squared error) averages squared errors, and its square root, **RMSE**, is back in price units but punishes large errors more heavily. **R²** is the share of price variance the model explains: 1 is perfect, and 0 means no better than always predicting the mean. Lower MAE, MSE and RMSE and higher R² are better.

12.1 Regression Evaluation

Table 9 and Figure 7 compare all six models. The model chosen by the pre-set rule is **Linear Regression on log(price)**, which has the lowest cross-validated RMSE (0.99M against 1.01M for the plain linear model and about 1.08M for the tree ensembles).

Model	CV RMSE	Train R ²	Test R ²	Test MAE	Test RMSE
Mean baseline	1.76	0.000	-0.018	1.75	2.27
Linear Regression	1.01	0.693	0.649	0.98	1.33
Linear Regression (log price)	0.99	0.704	0.659	0.97	1.31
Random Forest (default)	1.08	0.950	0.612	1.02	1.40
Random Forest (depth-limited)	1.10	0.764	0.583	1.09	1.45
Gradient Boosting	1.08	0.862	0.665	0.96	1.30

Table 9. Model Comparison (Errors in Millions)

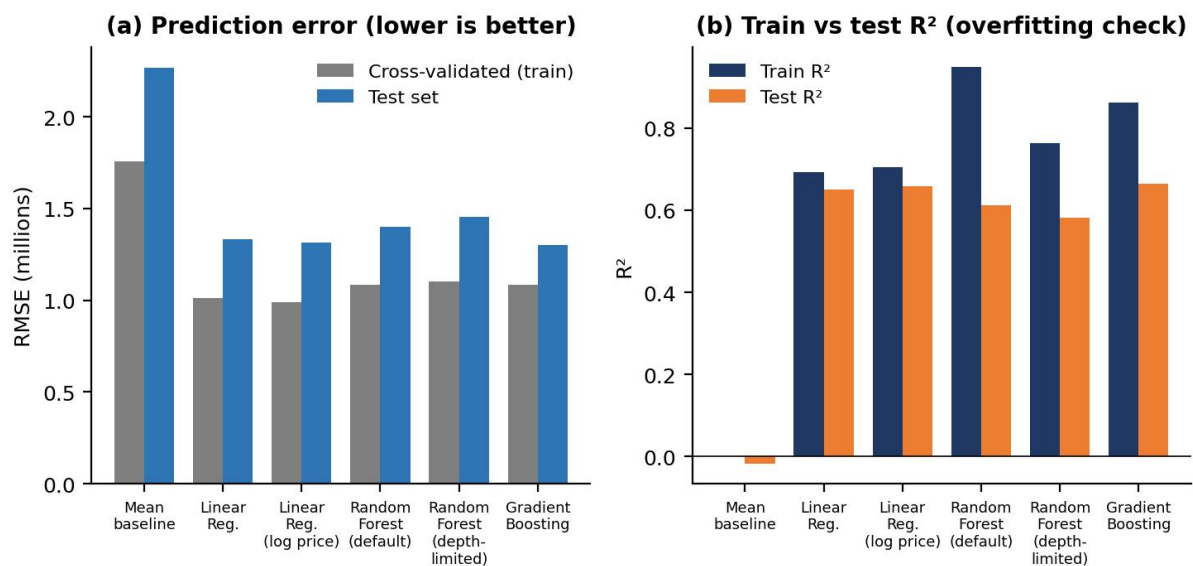


Figure 7. Model Comparison: Prediction Error (a) and Train vs Test R² (b)

Every real model is far better than the baseline (test RMSE about 1.31M versus 2.27M). Gradient Boosting has a marginally higher test R² (0.665 versus 0.659), but that difference is too small to draw a conclusion from on only 109 test homes, and Gradient Boosting was worse on cross-validation and shows a much

larger train-test gap. Because model choice was made on cross-validation and not on the test set, the comparison is not biased toward whichever model happened to score best on those 109 homes.

Table 10 gives the performance of the selected model in full.

Metric	Train	Test	What it means
MAE	0.68	0.97	On average the estimate is off by 0.97M, about 20% of the mean price
MSE (millions ²)	0.91	1.73	Average squared error; less intuitive, used to compute RMSE
RMSE	0.96	1.31	Typical error with extra weight on big misses; 28% of mean price
R ²	0.704	0.659	The model explains 65.9% of price variance on unseen homes

Table 10. Regression Performance of the Selected Model (Linear Regression on log Price)

Is this error acceptable? It depends on the use. About **60% of test predictions fall within 20% of the true price** and 27% within 10%; the median error is 16% of price. That is useful for screening listings, giving a first benchmark, and comparing homes, but not accurate enough to set the price of an individual transaction, where errors of a million or more would matter.

12.2 Overfitting and Underfitting

Comparing training and test performance (Figure 7b and Table 9) shows how well each model generalises. The selected model is reasonably well fitted: training R² is 0.704 and test R² 0.659, a gap of only 0.046, so it performs almost as well on new homes as on the homes it learned from. The default Random Forest is **clearly overfitted**: it fits the training data almost perfectly (R² 0.950) but drops to 0.612 on the test set, a gap of 0.338. Limiting its depth reduced the gap (0.182) but also lowered its test score (0.583). Gradient Boosting has a gap of 0.197. There is little sign of underfitting in the linear model: the more flexible models, despite having far more capacity, did not achieve a better cross-validated error. That suggests the ceiling on accuracy comes mainly from information the dataset does not contain (such as exact location and condition), not from the simplicity of the model.

12.3 Error Analysis

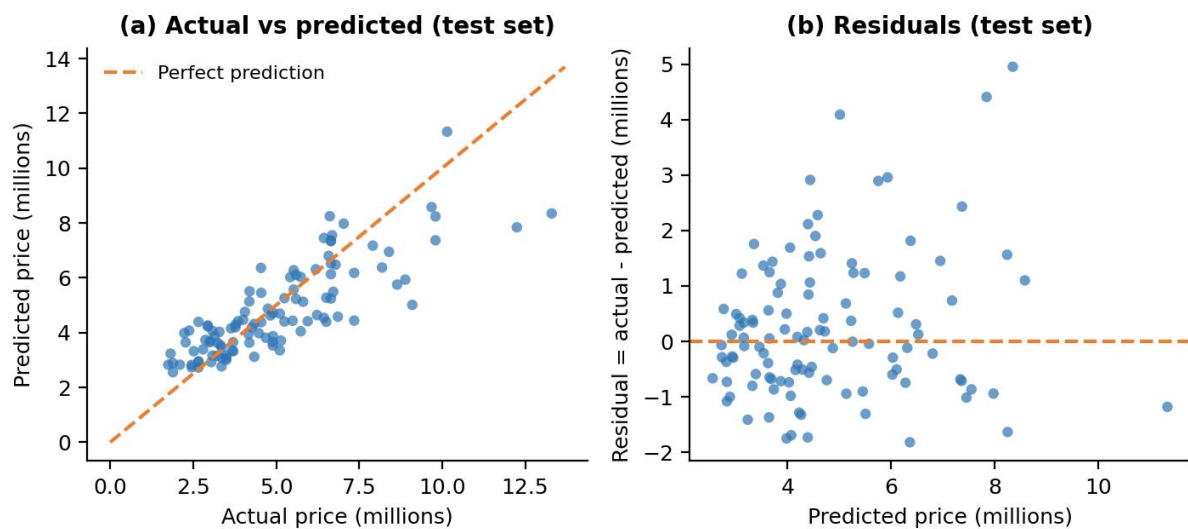


Figure 8. Actual versus Predicted Price (a) and Residuals (b) on the Test Set

Predictions track the diagonal well for low and mid-priced homes but fall increasingly below it for expensive ones (Figure 8a), and the residuals spread out as predicted price rises (Figure 8b). Table 11 confirms this by price band (bands defined from training prices). Errors are roughly balanced in the middle third, the model over-predicts the cheapest homes by 0.50M on average, and it **under-predicts the most expensive third by 0.97M on average**. Absolute error is about twice as large there (1.37M versus about 0.67M), although the median percentage error is similar across bands (17%, 13% and 16%). The five

largest errors in the test set (Appendix B, Table 13) are all under-predictions of homes priced between 7.35 and 13.30 million.

Price band	Homes	MAE (M)	Mean error (M)	Median % error
Lower third	39	0.66	-0.50	17.5%
Middle third	23	0.67	0.00	13.4%
Upper third	47	1.37	+0.97	16.0%

Table 11. Test-Set Error by Price Band

Mean error is actual minus predicted: a positive value means the model under-predicts. This behaviour is typical of regression models, which pull extreme values toward the average, and it is amplified here because the features cannot capture what makes a luxury home luxurious.

12.4 Interpreting the Model

To understand what drives price, a linear regression on $\log(\text{price})$ was fitted to the training data. Coefficients from a log-price model translate into approximate percentage effects, holding the other features constant (Table 12 and Figure 9). This interpretation model explains 72% of the variance of log price on the training data.

Feature	Estimated effect	Standardised coefficient	Rank by influence
log_area	elasticity 0.30	+0.118	1
bathrooms	+20.3%	+0.088	2
stories	+8.6%	+0.071	3
airconditioning	+15.7%	+0.067	4
prefarea	+14.3%	+0.056	5
furnishingstatus_unfurnished	-10.8%	-0.053	6
basement	+11.0%	+0.050	7
mainroad	+9.3%	+0.031	8
hotwaterheating	+14.1%	+0.029	9
parking	+3.4%	+0.028	10
bedrooms	+2.1%	+0.015	11
guestroom	+3.7%	+0.014	12
furnishingstatus_semi-furnished	-0.5%	-0.002	13

Table 12. Estimated Effect of Each Feature on Price (Log-Price Linear Model)

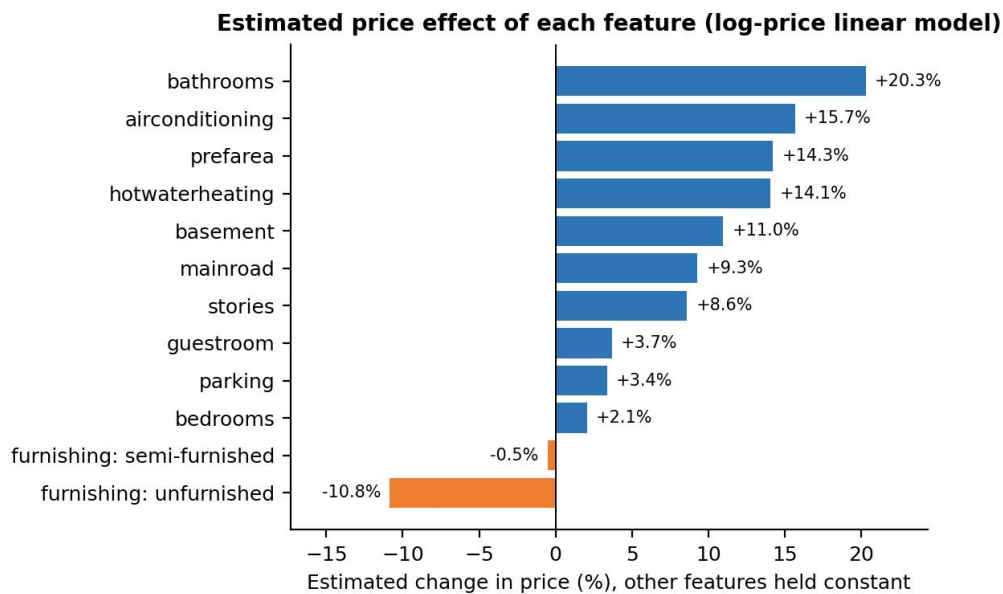


Figure 9. Estimated Percentage Change in Price Associated With Each Feature

Area has the largest standardised influence: a 10% larger home is associated with a price about 2.9% higher (elasticity 0.30). Each additional bathroom is associated with about 20% higher price and each additional storey with about 9%. Air conditioning (+16%), preferred-area location (+14%), hot-water heating (+14%, based on only 25 homes), basement (+11%) and main-road access (+9%) carry substantial premiums, while unfurnished homes are about 11% cheaper than furnished ones and semi-furnished homes are nearly identical to furnished ones (-0.5%). Notably, bedrooms add only about 2% once area and bathrooms are accounted for. The adjusted air-conditioning premium (+16%) is much smaller than the raw 48% gap in medians. These are associations estimated from a small sample without confidence intervals, so they indicate direction and rough size, not precise or causal effects.

13. RESULTS AND FINDINGS

The evidence answers the four research questions as follows.

- 1. The data is clean, but price is skewed.** There are no missing values, duplicates, inconsistent labels or impossible values (Table 4); the 15 price and 12 area outliers are genuine high-end homes and were retained. Price is right-skewed (skewness 1.21, Figure 1), which favours the median as a summary and a log transformation for modelling.
- 2. Size and bathrooms dominate the raw associations.** Area ($r = 0.54$) and bathrooms ($r = 0.52$) show the strongest correlations with price, followed by air conditioning (0.45) (Figure 6). Every yes/no feature is linked to a higher median price, with air conditioning showing the largest raw gap (48%) (Table 6).
- 3. A simple log-price linear model predicts best.** It explains 65.9% of price variance on unseen homes with an MAE of 0.97M (20% of the mean price) and about 60% of predictions within 20% of the true price (Table 10).
- 4. More flexible models did not help and tended to overfit.** The default Random Forest scores R^2 0.95 on training but 0.61 on test, while the selected model's gap is only 0.05 (Figure 7). The limit on accuracy therefore appears to lie in the data, not in the algorithm.
- 5. After adjusting for other features, area, bathrooms, storeys, air conditioning and preferred area matter most.** Adjusted premiums are considerably smaller than raw gaps; bedrooms contribute little once area and bathrooms are known (Table 12, Figure 9).

6. The model under-predicts expensive homes. For the upper price third the mean error is +0.97M and all five largest errors are under-predictions (Figure 8, Table 11). The model is least reliable exactly where prices are highest.

14. DISCUSSION

The results fit the story told by the exploratory analysis, and in several places refine it. The skew in price and area seen in Figure 1 and Figure 2, and the widening scatter in Figure 5, pointed to a multiplicative relationship in which a given feature changes price by a proportion rather than a fixed amount. The log-price model reflects that, and it was the best model on cross-validation, which is consistent with the EDA rather than a coincidence of the test split. Area was expected to dominate, and it does have the largest standardised influence, but bathrooms rank second and are almost as strongly correlated with price ($r = 0.52$ versus 0.54 for area), which is easy to overlook.

The comparison of raw and adjusted effects is the most instructive result. Air-conditioned homes have a median price 48% higher than the rest, but the model attributes only about 16% to air conditioning once size, bathrooms and other features are held constant. The rest of the gap plausibly reflects the fact that air-conditioned homes also tend to be larger and better equipped in other ways (air conditioning correlates positively with area, bathrooms and storeys in the correlation heatmap). Any decision based on raw group differences would therefore overestimate the benefit of a single feature. The same logic applies to bedrooms: they correlate with price ($r = 0.37$) partly because homes with more bedrooms also tend to have more bathrooms and storeys, and their independent contribution is small.

Why did a simple linear model beat Random Forest and Gradient Boosting? With 436 training homes and mostly binary or small-integer features, the relationships are largely additive and there is little data for tree models to learn interactions from; they memorise the training set instead (the default Random Forest's train R^2 of 0.95 versus test 0.61). The model's largest errors are in the priciest homes. There, price probably depends on things the dataset cannot see, such as precise location, build quality, finishes and view, and a model built on 12 coarse features can only regress toward the average. This is a data limitation more than a modelling failure.

A practitioner should take three things from this study. First, the model is a sound benchmark for typical homes, with a median error of about 16%. Second, its estimates for high-end homes should not be trusted without independent appraisal. Third, the feature effects describe associations in this sample; for instance, air conditioning may be a marker of an upscale build and not itself the cause of a higher price, so the fact that it is associated with a 16% premium does not mean installing it would raise a home's price by that amount.

15. LIMITATIONS

- **Small sample.** With 545 homes and a test set of only 109, all performance figures carry meaningful uncertainty, and a single random split was used. Small groups (25 homes with hot-water heating, one home with four bathrooms) give unreliable estimates.
- **Missing key variables.** The data has no precise location, property age, condition, build quality or sale date. Only two coarse location proxies (`mainroad`, `prefarea`) exist. This limits accuracy: about a third of price variance remains unexplained.
- **Unknown provenance.** The file does not state its source region, time period, currency or area unit. The results cannot be assumed to transfer to other markets or years, and prices cannot be interpreted in real monetary terms.
- **Model assumptions.** The selected model assumes additive effects on log price and works best when errors are of similar relative size; residuals do spread out at higher prices. Converting log predictions back to price units yields estimates that are typical rather than average, which may slightly understate the price of expensive homes.

- **Correlation, not causation.** All feature effects are observational associations. No experiment or causal design was used, and unmeasured factors are almost certainly mixed into the coefficients.
- **Limited model search.** Six models were compared with minimal tuning, deliberately, to avoid overfitting a small dataset. A wider search might improve performance slightly, though the evidence here suggests that data quality, not tuning, is the bottleneck.
- **Outliers retained.** Keeping genuine high-end homes preserves information but contributes to larger errors at the top of the market.

16. CONCLUSION

This project set out to predict house prices from the characteristics of 545 properties and to identify which characteristics matter most. The dataset was found to be complete and consistent, and its skewed price distribution guided the modelling. Six regression models were compared on a fair, leakage-free basis, and a linear regression on log-transformed price with a log-area feature was selected on cross-validated error. It explains 65.9% of price variance on unseen homes with a mean absolute error of 0.97 million (about 20% of the average price) and generalises well, with a train-test R^2 gap of only 0.05. Area, bathrooms, storeys, air conditioning and preferred-area location are the main associated drivers of price, and raw group differences overstate the effect of individual features. The model is a useful benchmark for typical homes but under-predicts high-end properties and cannot replace a professional valuation. The most promising route to better predictions is richer data, not a more complex algorithm.

17. RECOMMENDATIONS AND FUTURE WORK

17.1 Recommendations

Recommendation 1: Use the model as a screening benchmark, not as an appraisal. Finding: the model explains 66% of price variance, with a median error of 16% and about 60% of estimates within 20% of the true price. Implication: it is reliable enough to flag mispriced listings but not to fix a transaction price. Action: agents and lenders should treat listings whose asking price differs from the model estimate by clearly more than the typical error as candidates for closer review.

Recommendation 2: Do not use this model to value high-end homes. Finding: in the upper price third the model under-predicts by 0.97M on average and all five largest errors are luxury homes. Implication: the features available cannot capture what drives luxury prices. Action: value top-segment homes with comparable-sales analysis or a professional appraisal.

Recommendation 3: Prioritise size-related features when assessing value-adding improvements, and test before investing. Finding: after adjustment, an extra bathroom is associated with about 20% higher price and an extra storey with about 9%, versus only about 2% for a bedroom. Implication: bathrooms and usable floor space appear to be valued more than bedroom count. Action: developers and owners should compare the cost of such additions against these associated premiums, and pilot them locally, since the data shows association, not proven cause.

Recommendation 4: Improve the data before improving the algorithm. Finding: flexible models overfit and did not beat a simple one, and errors are concentrated where unobserved factors matter. Implication: accuracy is limited by missing information. Action: future data collection should record neighbourhood or coordinates, property age and condition, sale date, and the units and currency used.

17.2 Future Work

Future work should (1) collect richer variables, above all location, age and condition; (2) use repeated cross-validation to obtain more stable performance estimates on this small dataset; (3) test regularised linear models and models with interaction terms, which suit small samples better than deep trees; (4) produce prediction intervals so users see the uncertainty around each estimate; (5) fit a separate model, or a

quantile-based model, for the high-end segment; and (6) validate the model on a second dataset from another region or period before any practical deployment.

18. REFERENCES

- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
- India Data Research Academy. (2026a). *Capstone Project Guide 2026: Data Science & AI training program*. India Data Research Academy.
- India Data Research Academy. (2026b). *P_9_Housing.csv* [Dataset for Capstone Project 9: House Price Prediction]. IDRA Learning Management System, Study Material.
- McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61).
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Waskom, M. L. (2021). seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.

APPENDIX A. CODE

The complete, executable code is in the submitted notebook `Surname_Name_Capstone_Notebook.ipynb`. The excerpts below are copied from it and show the key steps in order.

A.1 Data Loading and Inspection

```
raw = pd.read_csv(DATA_PATH)
```

A.2 Data Cleaning

```
missing = raw.isna().sum()

# Inconsistent labels: leading/trailing spaces or mixed capitalisation would create fake categories
label_issues = int(sum((raw[c] != raw[c].str.strip().str.lower()).sum() for c in cat_cols))
for c in cat_cols:

# Validity: price, area, bedrooms, bathrooms, stories must be > 0; parking may legitimately be 0 (no parking) but not negative
positive_cols = ["price", "area", "bedrooms", "bathrooms", "stories"]
invalid = int((raw[positive_cols] <= 0).sum().sum() + (raw["parking"] < 0).sum())

# Sanity check on the price-per-area ratio (catches impossible price/area combinations)
ppa = raw["price"] / raw["area"]
```

```
df = raw.copy()
for c in cat_cols:
    df[c] = df[c].str.strip().str.lower().astype("category") # standardise text, store as categorical

after = dict(rows=len(df), columns=df.shape[1], missing_cells=int(df.isna().sum().sum()),
            duplicate_rows=int(df.duplicated().sum()),
            label_issues=int(sum((df[c].astype(str) !=
```

```
df[c].astype(str).str.strip().str.lower()).sum() for c in cat_cols)),
    invalid_numeric=int((df[positive_cols] <= 0).sum().sum() + (df["parking"] <
0).sum()),
    categorical_columns=int(df.select_dtypes("category").shape[1]))

# Validation: prove the dataset is in the state we claim
assert after["rows"] == R["before"]["rows"]
assert after["missing_cells"] == 0 and after["duplicate_rows"] == 0 and
after["invalid_numeric"] == 0
```

A.3 Preprocessing

```
data = df.copy()
for c in binary_cols:
    data[c] = (data[c] == "yes").astype(int)
data = pd.get_dummies(data, columns=["furnishingstatus"], drop_first=True, dtype=int)
```

A.4 EDA and Visualization

```
price_m = df["price"] / 1e6

fig, ax = plt.subplots(1, 2, figsize=(6.6, 2.9))
ax[0].hist(price_m, bins=30, color=BLUE, edgecolor="white")
ax[0].axvline(price_m.mean(), color=ORANGE, ls="--", label=f"Mean {price_m.mean():.2f}")
ax[0].axvline(price_m.median(), color=NAVY, ls="-", label=f"Median
{price_m.median():.2f}")
ax[0].set(title="Price", xlabel="Price (millions)", ylabel="Number of homes");
ax[0].legend(frameon=False)
ax[1].hist(np.log(df.price), bins=30, color=BLUE, edgecolor="white")
ax[1].set(title="log(Price)", xlabel="log(price)", ylabel="Number of homes")
save_fig("fig1_price_distribution")
```

A.5 Model Training and Evaluation

```
kf = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

def cv_rmse(model, X_, y_):
    return -cross_val_score(model, X_, y_, cv=kf,
scoring="neg_root_mean_squared_error").mean()

def make_linear(log_target):
    pipe = make_pipeline(StandardScaler(), LinearRegression()) # scaler is fitted inside
each CV fold / on train only
    if log_target:
        return TransformedTargetRegressor(regressor=pipe, func=np.log,
inverse_func=np.exp)
    return pipe

def add_candidates(X_):
    Z = X_.copy()
    Z["log_area"] = np.log(Z["area"])
    Z["amenity_score"] = Z[["guestroom", "basement", "hotwaterheating",
"airconditioning"]].sum(axis=1)
    Z["area_per_bedroom"] = Z["area"] / Z["bedrooms"]
    Z["bath_per_bedroom"] = Z["bathrooms"] / Z["bedrooms"]
    return Z

Xc = add_candidates(X_train)
base = list(X_train.columns)
trials = {
    "Baseline (13 original features)": base,
    "+ log_area": base + ["log_area"],
    "log_area replaces area": [c for c in base if c != "area"] + ["log_area"],
```

```

    "+ amenity_score":          base + ["amenity_score"],
    "+ area_per_bedroom":      base + ["area_per_bedroom"],
    "+ bath_per_bedroom":      base + ["bath_per_bedroom"],
}
fe_rows = [dict(feature_set=k, cv_rmse=cv_rmse(make_linear(True), Xc[v], y_train)) for k,
v in trials.items()]
fe = pd.DataFrame(fe_rows)
fe["change_vs_baseline_pct"] = 100 * (fe.cv_rmse / fe.cv_rmse.iloc[0] - 1)

```

```

models = {
    "Mean baseline":          DummyRegressor(strategy="mean"),
    "Linear Regression":      make_linear(False),
    "Linear Regression (log price)": make_linear(True),
    "Random Forest (default)": RandomForestRegressor(random_state=RANDOM_STATE),
    "Random Forest (depth-limited)": RandomForestRegressor(n_estimators=300,
max_depth=6, min_samples_leaf=5, random_state=RANDOM_STATE),
    "Gradient Boosting":      GradientBoostingRegressor(random_state=RANDOM_STATE),
}

def metrics(model, X_, y_):
    p = model.predict(X_); mse = mean_squared_error(y_, p)
    return dict(MAE=mean_absolute_error(y_, p), MSE=mse, RMSE=float(np.sqrt(mse)),
R2=r2_score(y_, p))

results = []
for name, m in models.items():
    cv = cv_rmse(m, X_train_f, y_train)          # cross-validation on TRAINING data only
    m.fit(X_train_f, y_train)
    tr, te = metrics(m, X_train_f, y_train), metrics(m, X_test_f, y_test)
    results.append(dict(model=name, cv_rmse=cv,
                        train_MAE=tr["MAE"], train_RMSE=tr["RMSE"], train_R2=tr["R2"],
                        test_MAE=te["MAE"], test_MSE=te["MSE"], test_RMSE=te["RMSE"],
test_R2=te["R2"]))
res = pd.DataFrame(results)
res["r2_gap"] = res.train_R2 - res.test_R2

candidates = res[res.model != "Mean baseline"]
best_name = candidates.loc[candidates.cv_rmse.idxmin(), "model"]

```

```

best = models[best_name]
pred = best.predict(X_test_f)
res_test = metrics(best, X_test_f, y_test)
res_train = metrics(best, X_train_f, y_train)
mean_price = y.mean()
pct_err = np.abs(pred - y_test) / y_test

```

APPENDIX B. ADDITIONAL FIGURES AND TABLES

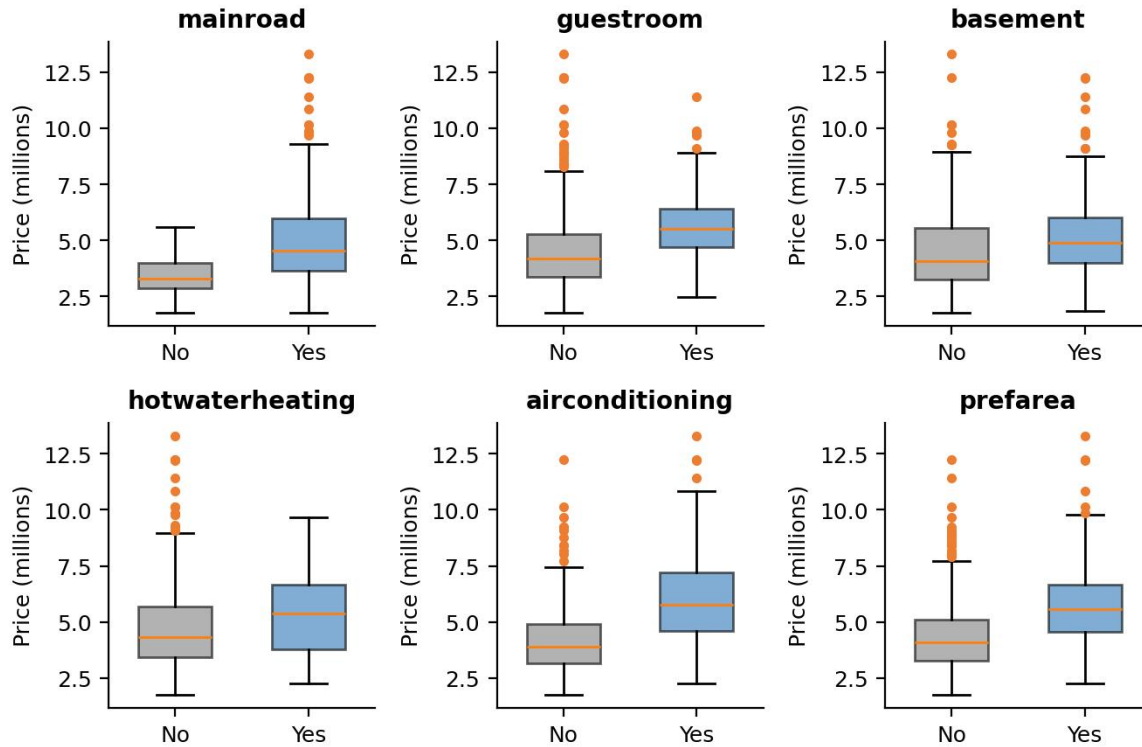


Figure 10. Box Plots of Price by Each Yes/No Feature

Figure 10 shows the spread of prices with and without each feature. The boxes overlap substantially, which explains why no single feature predicts price well on its own, and the orange points show that high-priced outliers occur in both groups for every feature.

Actual	Predicted	Error	Area	Bedrooms	Bathrooms	Stories	Air cond.
13.30	8.35	+4.95	7,420	4	2	3	Yes
12.25	7.84	+4.41	9,960	3	2	2	No
9.10	5.01	+4.09	6,000	4	1	2	No
8.89	5.93	+2.96	4,600	3	2	2	Yes
7.35	4.44	+2.91	6,000	3	1	2	Yes

Table 13. The Five Largest Prediction Errors on the Test Set (Prices in Millions)

Error is actual minus predicted; every one of the five is an under-prediction of a high-priced home.

Model	Train MAE	Train RMSE	Train R ²	Test MAE	Test RMSE	Test R ²
Mean baseline	1.35	1.76	0.000	1.75	2.27	-0.018
Linear Regression	0.71	0.97	0.693	0.98	1.33	0.649
Linear Regression (log price)	0.68	0.96	0.704	0.97	1.31	0.659
Random Forest (default)	0.28	0.39	0.950	1.02	1.40	0.612
Random Forest (depth-limited)	0.60	0.85	0.764	1.09	1.45	0.583
Gradient Boosting	0.48	0.65	0.862	0.96	1.30	0.665

Table 14. Training and Test Metrics for All Models (Errors in Millions)